

Recordkeeper Backup And Recovery Guide

This guide explains how to configure automatic PostgreSQL backups for the LAN deployment and how to restore the database if records are accidentally or maliciously deleted.

The commands below assume the production database is:

recordkeeper

and the database URL is:

postgresql://postgres:postgres@localhost:5432/recordkeeper

If your server uses a different PostgreSQL username, password, host, port, or database name, replace the URL everywhere in this guide.

1. What The Backup System Will Do

The recommended setup will:

- create one database backup every day at a fixed time
- store backups in '/opt/recordkeeper/backups'
- keep only the latest 7 daily backups
- write a small log file so backup success or failure can be checked
- allow recovery by restoring one selected backup into PostgreSQL

This backs up the PostgreSQL database only. It does not back up operating system files, Nginx config, app build files, or uploaded documents if those are added later.

2. Create The Backup Folder

Run this on the Ubuntu LAN server:

```
sudo mkdir -p /opt/recordkeeper/backups
sudo chown -R $USER:$USER /opt/recordkeeper/backups
chmod 700 /opt/recordkeeper/backups
```

The 'chmod 700' command restricts access to the backup folder because database backups may contain sensitive office data.

3. Create The Backup Script

Create a script file:

```
nano /opt/recordkeeper/backup-recordkeeper.sh
```

Paste this:

```
#!/bin/bash
set -euo pipefail

BACKUP_DIR="/opt/recordkeeper/backups"
DB_URL="postgresql://postgres:postgres@localhost:5432/recordkeeper"
DATE="$(date +%F_%H-%M-%S)"
BACKUP_FILE="$BACKUP_DIR/recordkeeper_$DATE.sql"
LOG_FILE="$BACKUP_DIR/backup.log"

mkdir -p "$BACKUP_DIR"

echo "[$(date '+%F %T')] Starting backup: $BACKUP_FILE" >> "$LOG_FILE"

pg_dump "$DB_URL" > "$BACKUP_FILE"

gzip "$BACKUP_FILE"

find "$BACKUP_DIR" -name "recordkeeper_*.sql.gz" -type f -mtime +6 -delete

echo "[$(date '+%F %T')] Backup completed successfully" >> "$LOG_FILE"
```

Save and close the file.

Make the script executable:

```
chmod +x /opt/recordkeeper/backup-recordkeeper.sh
```

4. Test The Backup Script Manually

Before scheduling it, run it once:

```
/opt/recordkeeper/backup-recordkeeper.sh
```

Check that a backup file was created:

```
ls -lh /opt/recordkeeper/backups
```

You should see a file like:

```
recordkeeper_2026-06-24_20-00-00.sql.gz
```

Check the log:

```
tail -n 20 /opt/recordkeeper/backups/backup.log
```

If the script fails, fix the database URL or PostgreSQL permissions before continuing.

5. Schedule Daily Automatic Backups

Open the current user's crontab:

```
crontab -e
```

Add this line to run the backup every day at 8:00 PM:

```
0 20 * * * /opt/recordkeeper/backup-recordkeeper.sh >> /opt/recordkeeper/backups/cron.log 2>&1
```

Save and close.

To choose a different time, change the first two values:

```
minute hour * * *
```

Examples:

```
# Every day at 1:30 PM
30 13 * * * /opt/recordkeeper/backup-recordkeeper.sh >> /opt/recordkeeper/backups/cron.log 2>&1
```

```
# Every day at 11:00 PM
0 23 * * * /opt/recordkeeper/backup-recordkeeper.sh >> /opt/recordkeeper/backups/cron.log 2>&1
```

6. Confirm The Schedule Is Installed

Run:

```
crontab -l
```

Confirm the backup line is visible.

After the scheduled time passes, check:

```
ls -lh /opt/recordkeeper/backups
tail -n 50 /opt/recordkeeper/backups/backup.log
tail -n 50 /opt/recordkeeper/backups/cron.log
```

7. How Retention Works

This line in the script removes old backups:

```
find "$BACKUP_DIR" -name "recordkeeper_*.sql.gz" -type f -mtime +6 -delete
```

Because the backup runs daily, this keeps roughly the latest 7 daily backups.

If you want to keep 14 days instead, change '+6' to '+13'.

If you want to keep 30 days, change '+6' to '+29'.

8. Recommended Extra Protection

Daily backups are helpful, but they are still on the same server. For better protection:

- copy backups to another computer or external drive regularly
- keep at least one weekly backup outside the server
- restrict access to '/opt/recordkeeper/backups'
- restrict who has admin access inside the app
- periodically test restoring a backup

If someone deletes data before the daily backup runs, restoring the previous backup may lose legitimate work done after that backup. For busy offices, consider two backups per day.

Example for 1:00 PM and 8:00 PM daily:

```
0 13 * * * /opt/recordkeeper/backup-recordkeeper.sh >> /opt/recordkeeper/backups/cron.log 2>&1
0 20 * * * /opt/recordkeeper/backup-recordkeeper.sh >> /opt/recordkeeper/backups/cron.log 2>&1
```

9. Recovery Scenario: Data Was Deleted

If someone signs in and deletes important data, act quickly.

Step 1: Stop The Backend

Stop the app backend so users cannot continue changing data during recovery:

```
sudo systemctl stop recordkeeper-backend
```

Nginx can remain running, but the app will not work properly until the backend is started again.

Step 2: Save The Damaged Database

Before restoring, save the current damaged database. This may help with investigation or manual recovery of records entered after the last good backup.

```
pg_dump "postgresql://postgres:postgres@localhost:5432/recordkeeper" > /opt/recordkeeper/backups/damaged_recordkeeper_$(date +%F_%H-%M-%S).sql
```

Step 3: Choose The Backup To Restore

List available backups:

```
ls -lh /opt/recordkeeper/backups/recordkeeper_*.sql.gz
```

Choose the latest backup from before the deletion happened.

Example:

```
/opt/recordkeeper/backups/recordkeeper_2026-06-24_20-00-00.sql.gz
```

Step 4: Drop And Recreate The Database

This removes the damaged production database and creates a clean empty database.

```
sudo -u postgres dropdb recordkeeper  
sudo -u postgres createdb recordkeeper
```

Only run these commands after you have selected the backup and saved the damaged database.

Step 5: Restore The Selected Backup

Restore from the compressed backup:

```
gunzip -c /opt/recordkeeper/backups/recordkeeper_2026-06-24_20-00-00.sql.gz |  
psql "postgres://postgres:postgres@localhost:5432/recordkeeper"
```

Replace the filename with the backup you selected.

Step 6: Start The Backend

```
sudo systemctl start recordkeeper-backend  
sudo systemctl status recordkeeper-backend
```

Step 7: Verify The App

Open the LAN app in a browser and check:

- login works
- dashboard loads
- search shows expected records
- reports load
- add/edit workflow works
- admin settings load

Also check the backend health endpoint:

```
curl http://localhost:3000/api/health
```

10. Important Recovery Notes

Restoring a backup returns the database to the exact state at the backup time.

Example:

- backup time: 8:00 PM yesterday
- deletion time: 4:00 PM today
- restored backup: 8:00 PM yesterday

In this case, all legitimate work entered between 8:00 PM yesterday and 4:00 PM today will not be present after restore.

To preserve evidence or recover later entries manually, keep the damaged dump created in Step 2.

11. Test Restore Without Touching Production

A backup is only useful if it can be restored. Periodically test one backup into a separate test database:

```
sudo -u postgres dropdb --if-exists recordkeeper_restore_test
sudo -u postgres createdb recordkeeper_restore_test
gunzip -c /opt/recordkeeper/backups/recordkeeper_2026-06-24_20-00-00.sql.gz |
psql "postgresql://postgres:postgres@localhost:5432/recordkeeper_restore_test"
```

Check tables exist:

```
psql "postgresql://postgres:postgres@localhost:5432/recordkeeper_restore_test"
-c "\dt"
```

When finished:

```
sudo -u postgres dropdb recordkeeper_restore_test
```

12. Quick Emergency Checklist

Use this checklist during an actual deletion incident:

1. Tell users to stop using the app.
2. Stop backend: `sudo systemctl stop recordkeeper-backend`
3. Dump damaged DB for investigation.
4. Pick latest good backup before deletion.
5. Drop and recreate recordkeeper database.
6. Restore selected backup.
7. Start backend.

8. Verify login, search, dashboard, reports, add/edit.
9. Preserve damaged dump until investigation is complete.